# Memory Hierarchy & Cache Memories

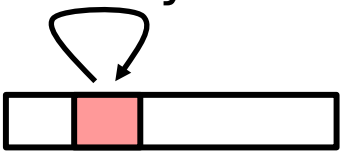
CSE 238/2038/2138: Systems Programming

**Instructor:**
Zuhal ALTUNTAŞ & Birol GENÇYILMAZ

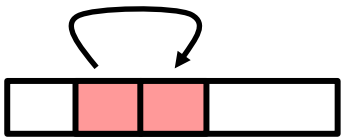
*Slides adapted from Bryant & O'Hallaron's slides*

# Locality

- Principle of Locality: Programs tend to use data and instructions with addresses near or equal to those they have used recently



- Temporal locality:
  - Recently referenced items are likely to be referenced again in the near future



- Spatial locality:
  - Items with nearby addresses tend to be referenced close together in time

# Locality Example

```
sum = 0;
for (i = 0; i < n; i++)
     sum += a[i];
return sum;
```

- Data references
  1. Reference array elements sequentially (stride-1 reference pattern). — Spatial locality
  2. Reference variable `sum` each iteration. — Temporal locality
- Instruction references
  1. Reference instructions in sequence. — Spatial locality
  2. Cycle through loop repeatedly. — Temporal locality

# Locality Example

```
1  int sumarrayrows(int a[M][N])
2  {
3      int i, j, sum = 0;
4
5      for (i = 0; i < M; i++)
6          for (j = 0; j < N; j++)
7              sum += a[i][j];
8      return sum;
9  }
```

(a)

| Address | 0 | 4 | 8 | 12 | 16 | 20 |
|---|---|---|---|---|---|---|
| Contents | $a_{00}$ | $a_{01}$ | $a_{02}$ | $a_{10}$ | $a_{11}$ | $a_{12}$ |
| Access order | 1 | 2 | 3 | 4 | 5 | 6 |

(b)

**Figure 6.18** **(a) Another function with good locality. (b) Reference pattern for array a ($M = 2$, $N = 3$).** There is good spatial locality because the array is accessed in the same row-major order in which it is stored in memory.

```
1  int sumarraycols(int a[M][N])
2  {
3      int i, j, sum = 0;
4
5      for (j = 0; j < N; j++)
6          for (i = 0; i < M; i++)
7              sum += a[i][j];
8      return sum;
9  }
```

(a)

| Address | 0 | 4 | 8 | 12 | 16 | 20 |
|---|---|---|---|---|---|---|
| Contents | $a_{00}$ | $a_{01}$ | $a_{02}$ | $a_{10}$ | $a_{11}$ | $a_{12}$ |
| Access order | 1 | 3 | 5 | 2 | 4 | 6 |

(b)

**Figure 6.19** **(a) A function with poor spatial locality. (b) Reference pattern for array a ($M = 2$, $N = 3$).** The function has poor spatial locality because it scans memory with a stride-$N$ reference pattern.

# What Type of Locality?

The following function exhibits which type of locality? Consider *only* array accesses.

```
void who(int *arr, int size) {
  for (int i = 0; i < size-1; ++i)
    arr[i] = arr[i+1];
}
```

| | |
|---|---|
| **A.** | Spatial |
| **B.** | Temporal |
| **C.** | Both A and B |
| **D.** | Neither A nor B |

# What Type of Locality?

The following function exhibits which type of locality? Consider *only* array accesses.

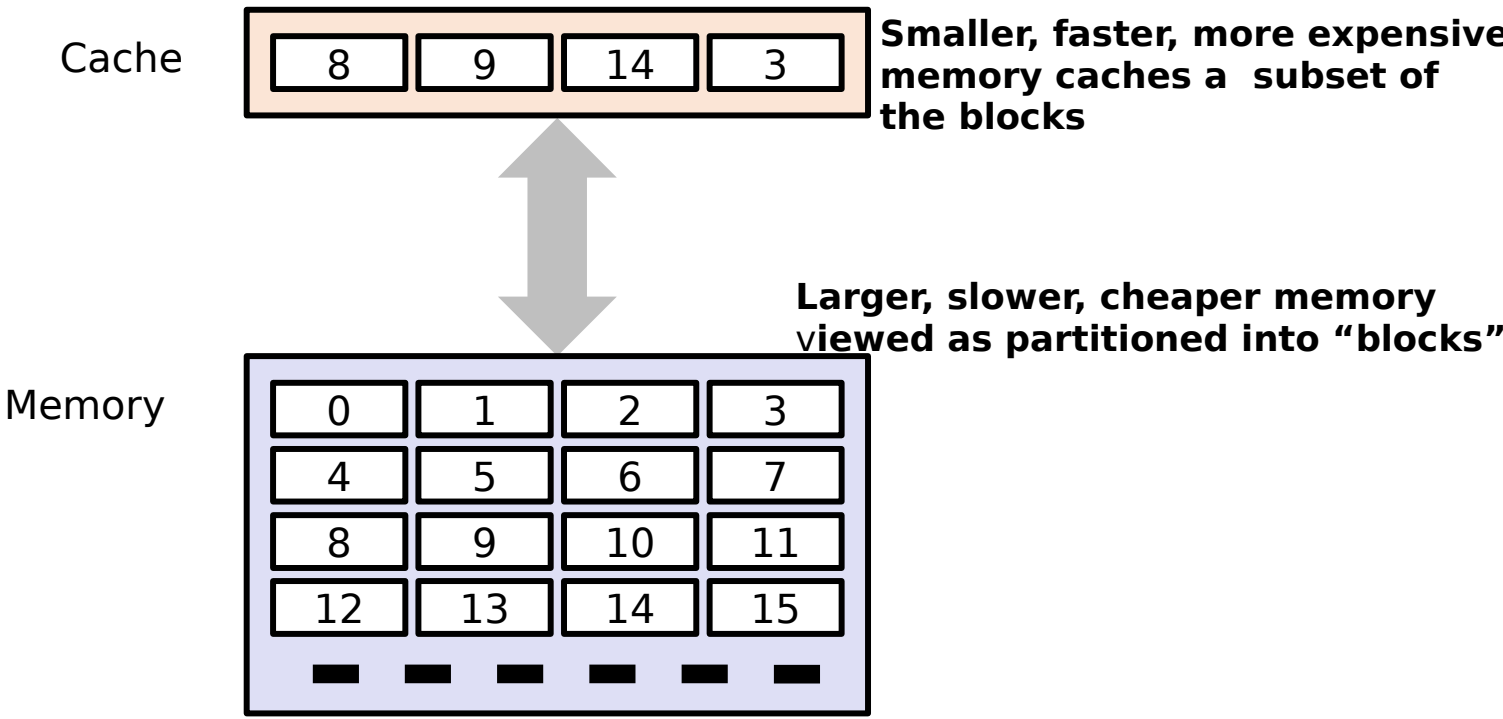
```
void coo(int *arr, int size) {
  for (int i = size-2; i >= 0; --i)
    arr[i] = arr[i+1];
}
```

| | |
|---|---|
| **A.** | Spatial |
| **B.** | Temporal |
| **C.** | Both A and B |
| **D.** | Neither A nor B |

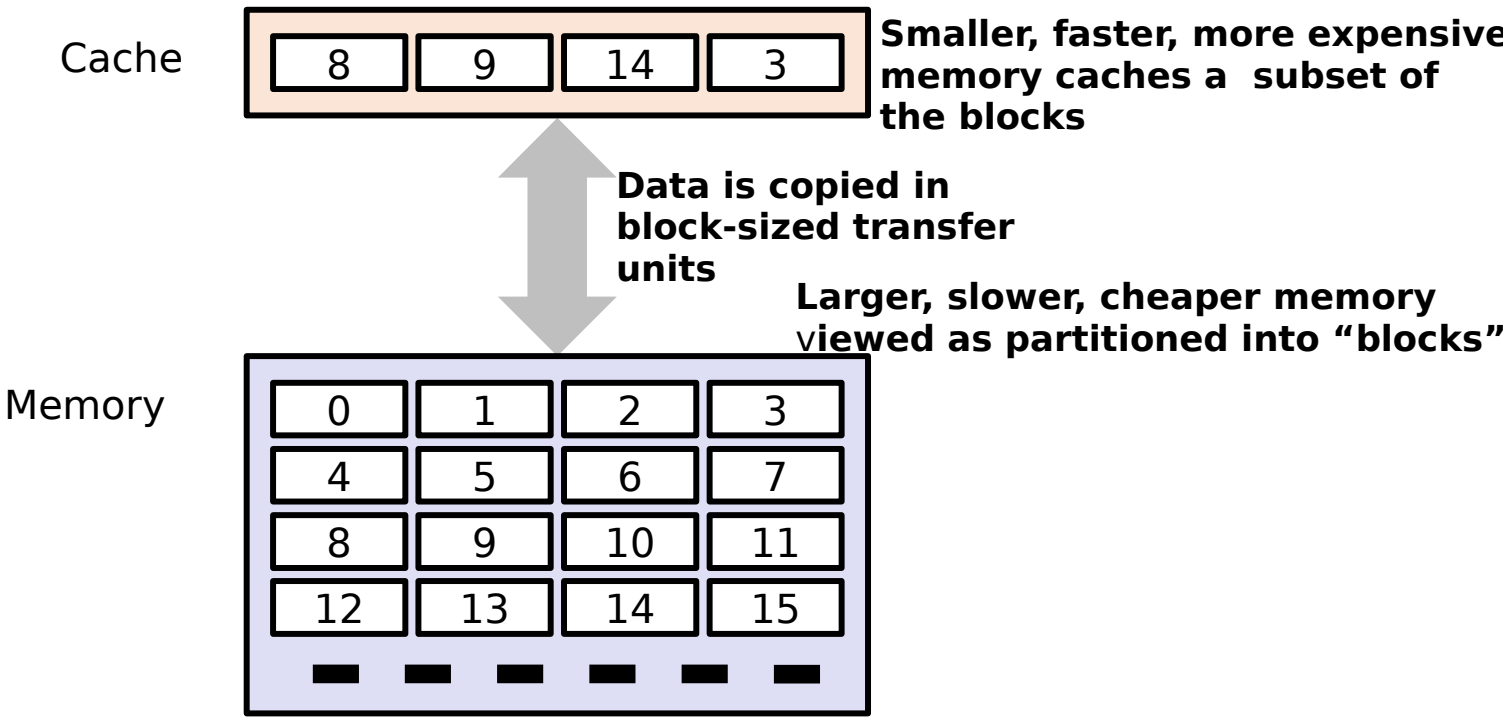
# Caches

- *Cache:* A smaller, faster storage device that acts as a staging area for a subset of the data in a larger, slower device.
- Fundamental idea of a memory hierarchy:
  - For each k, the faster, smaller device at level k serves as a cache for the larger, slower device at level k+1.
- Why do memory hierarchies work?
  - Because of locality, programs tend to access the data at level k more often than they access the data at level k+1.
  - Thus, the storage at level k+1 can be slower, and thus larger and cheaper per bit.
- *Big Idea:* The memory hierarchy creates a large pool of storage that costs as much as the cheap storage near the bottom, but that serves data to programs at the rate of the fast storage near the top.

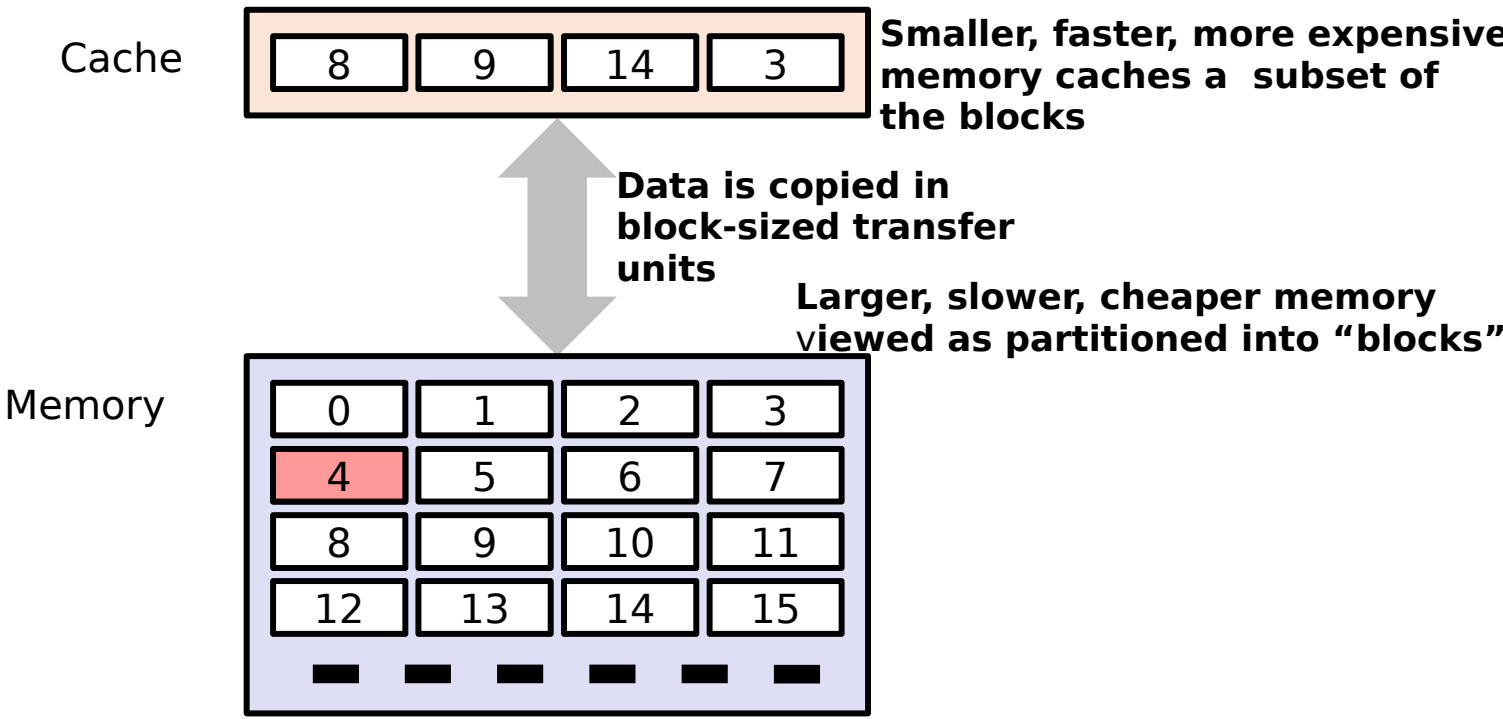
# General Cache Concepts



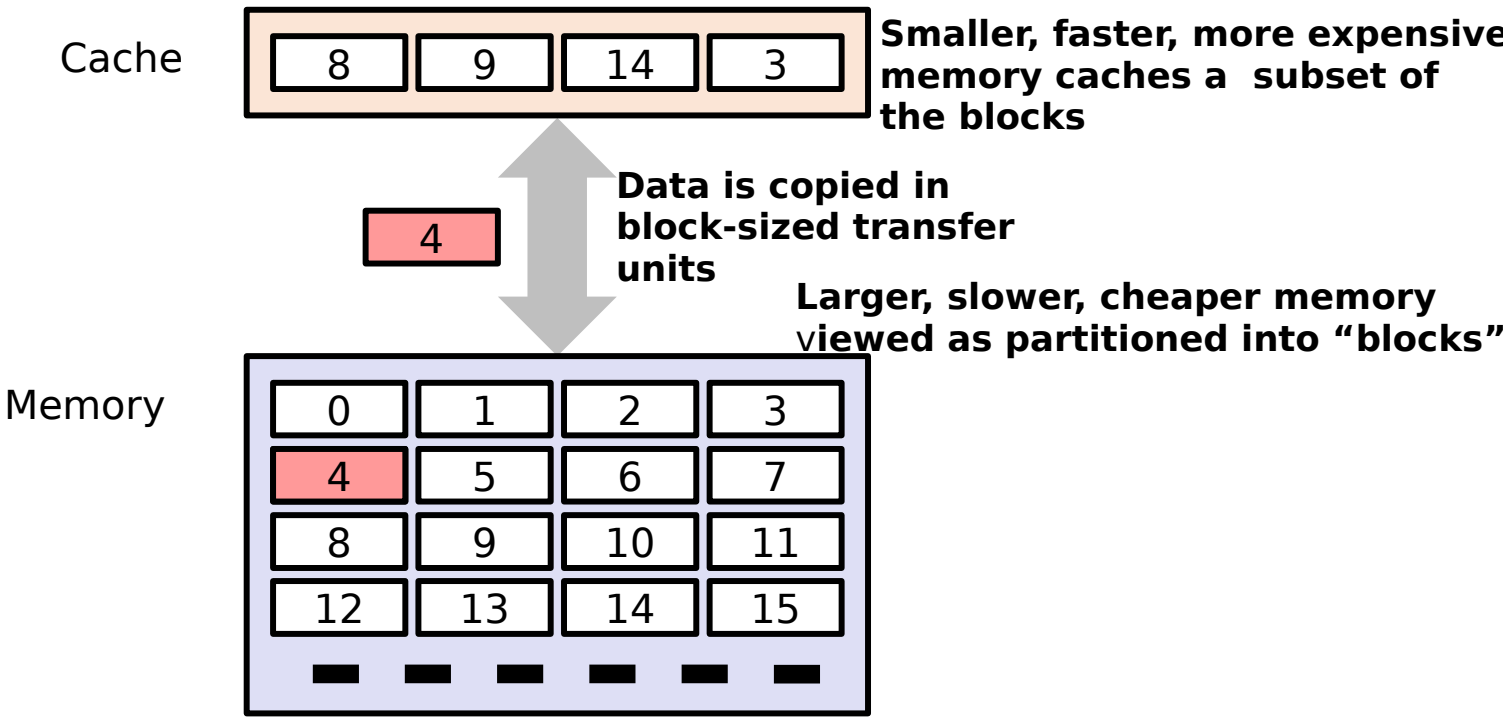
# General Cache Concepts



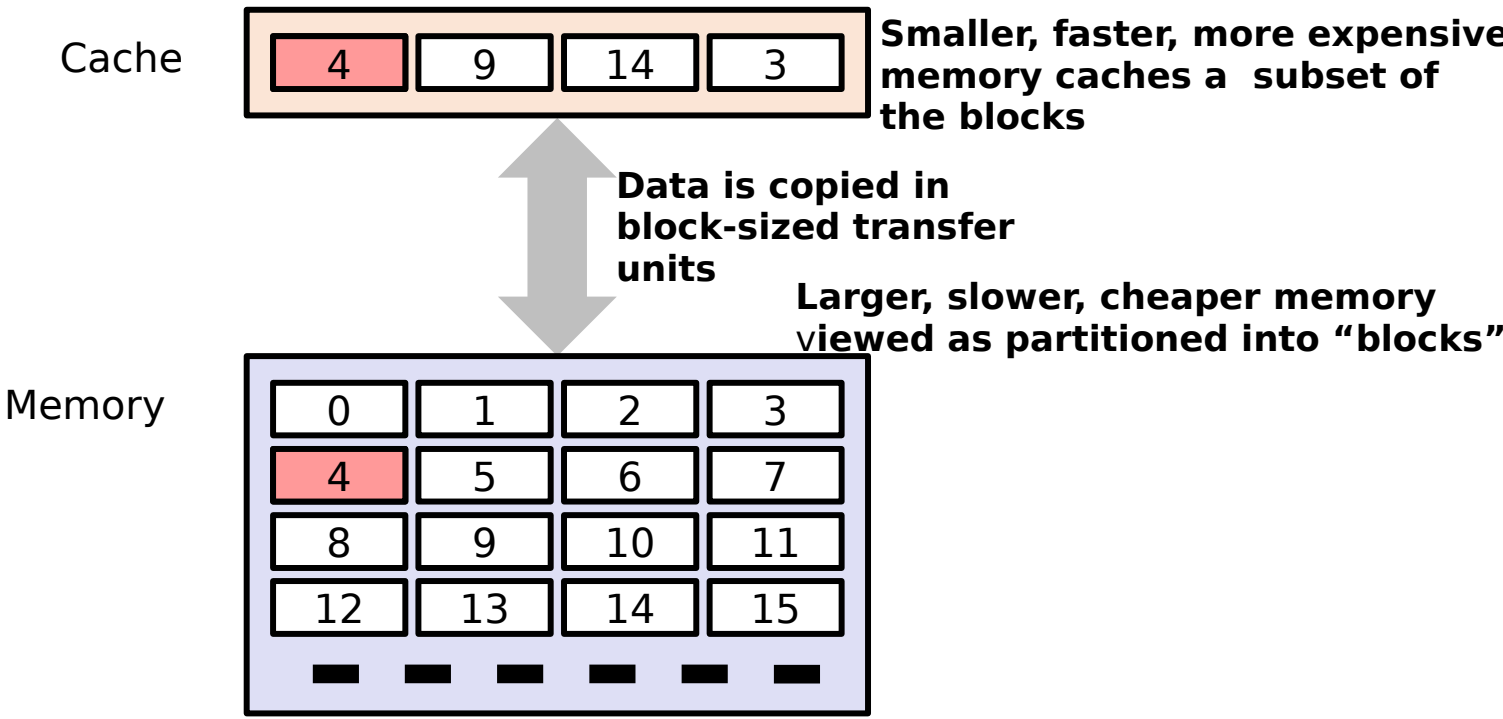
# General Cache Concepts



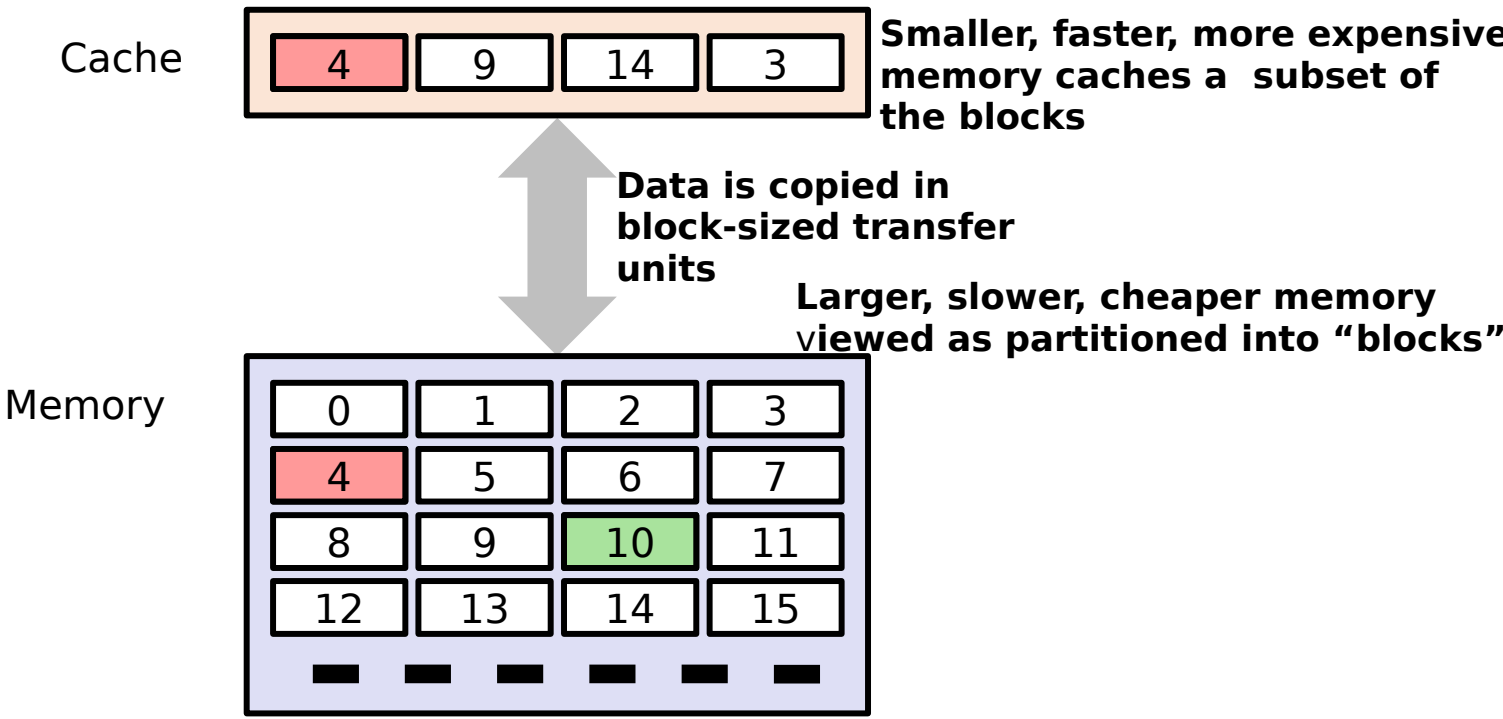
# General Cache Concepts



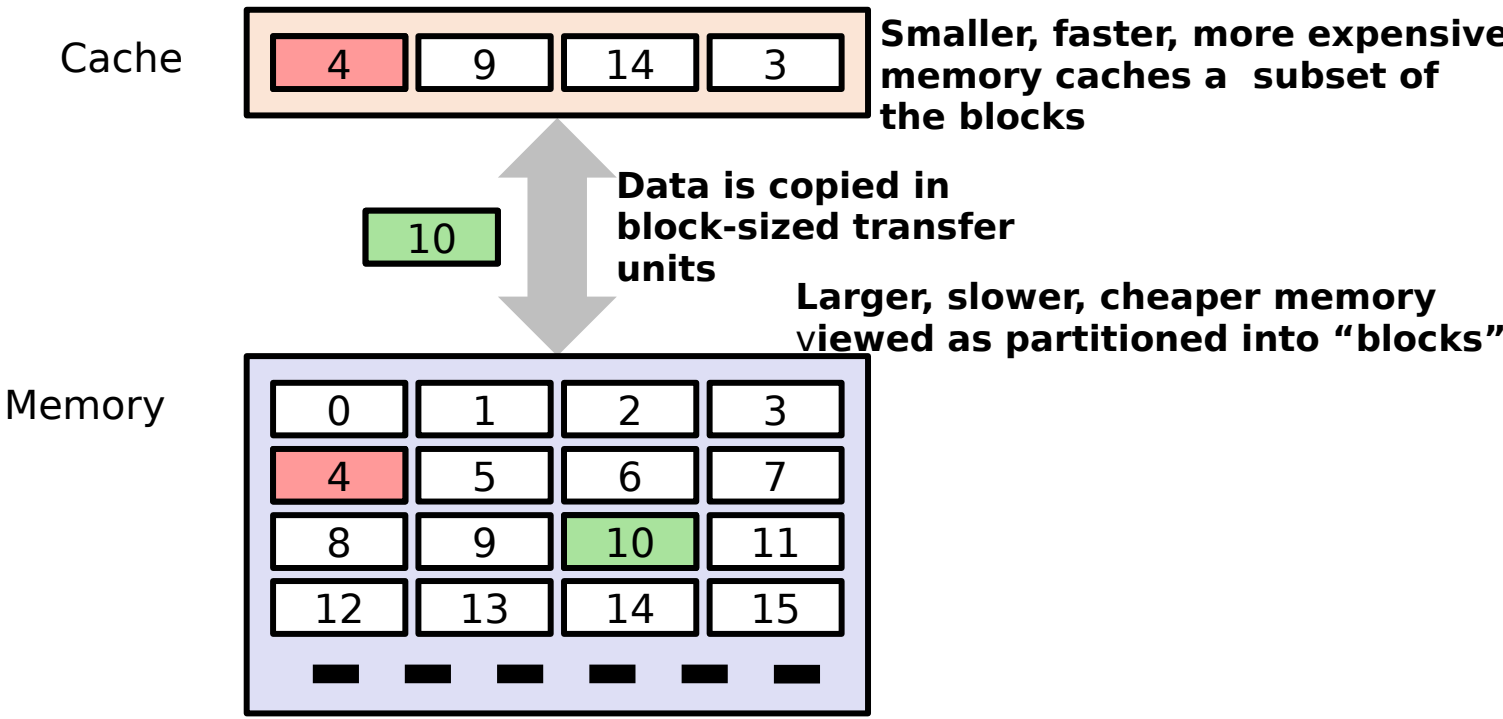
# General Cache Concepts



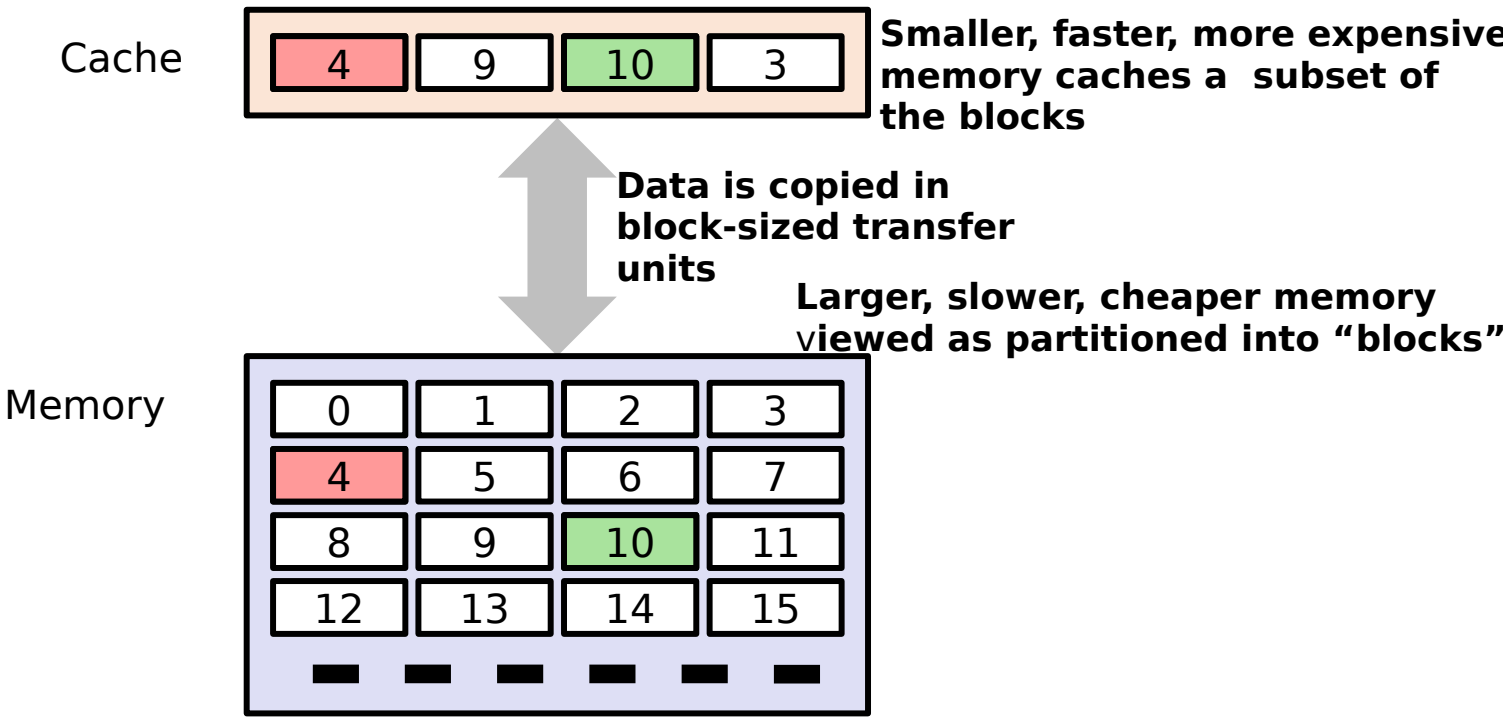
# General Cache Concepts



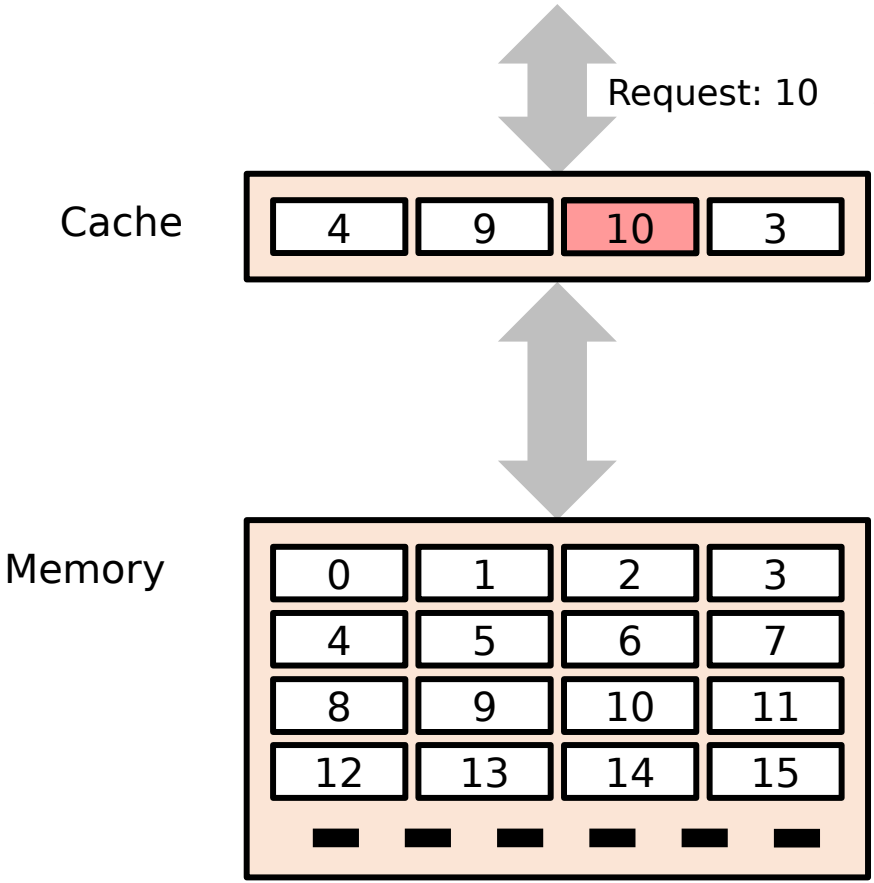
# General Cache Concepts



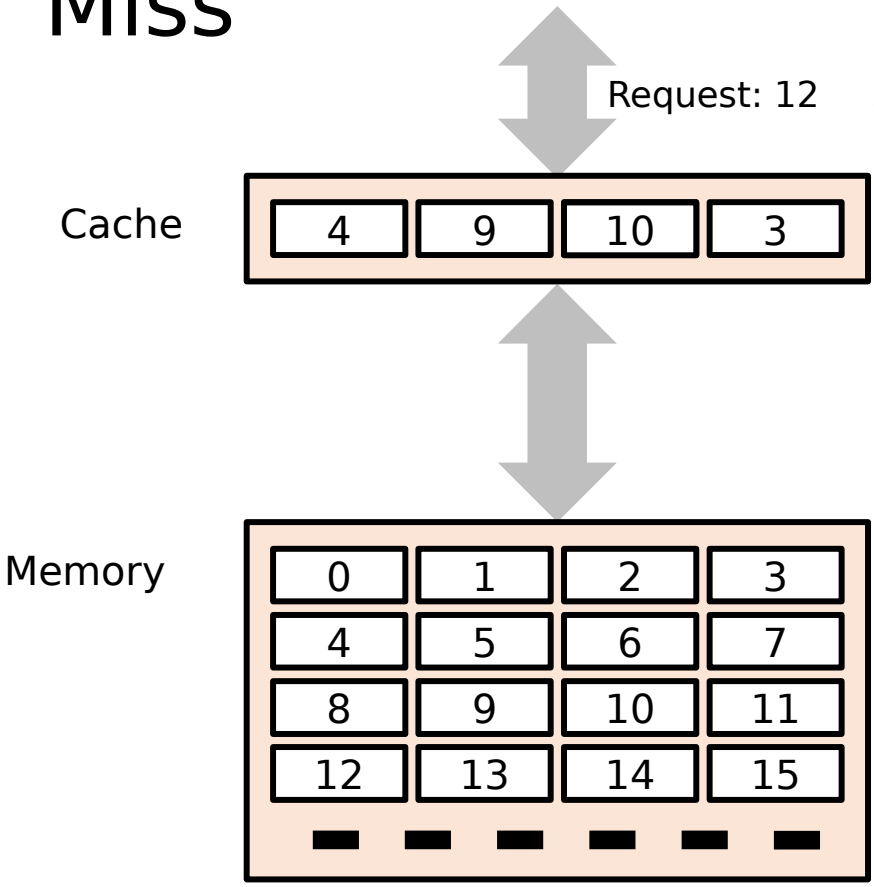
# General Cache Concepts



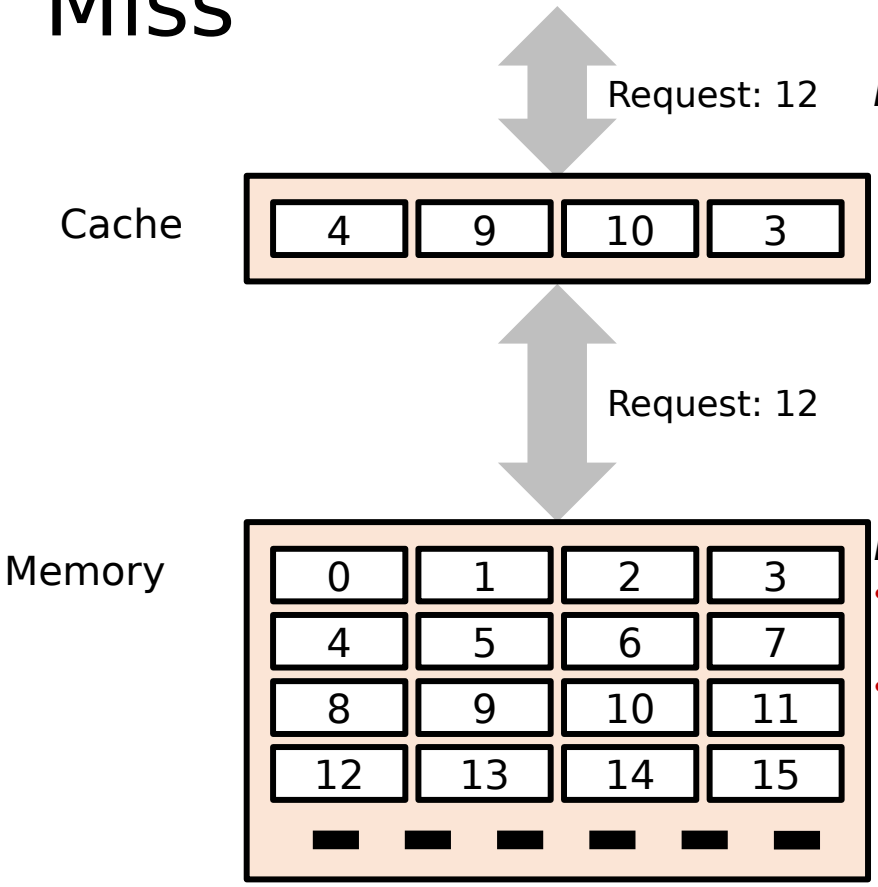
# General Cache Concepts: Hit



# General Cache Concepts: Miss



# General Cache Concepts: Miss




***Data in block b is needed***

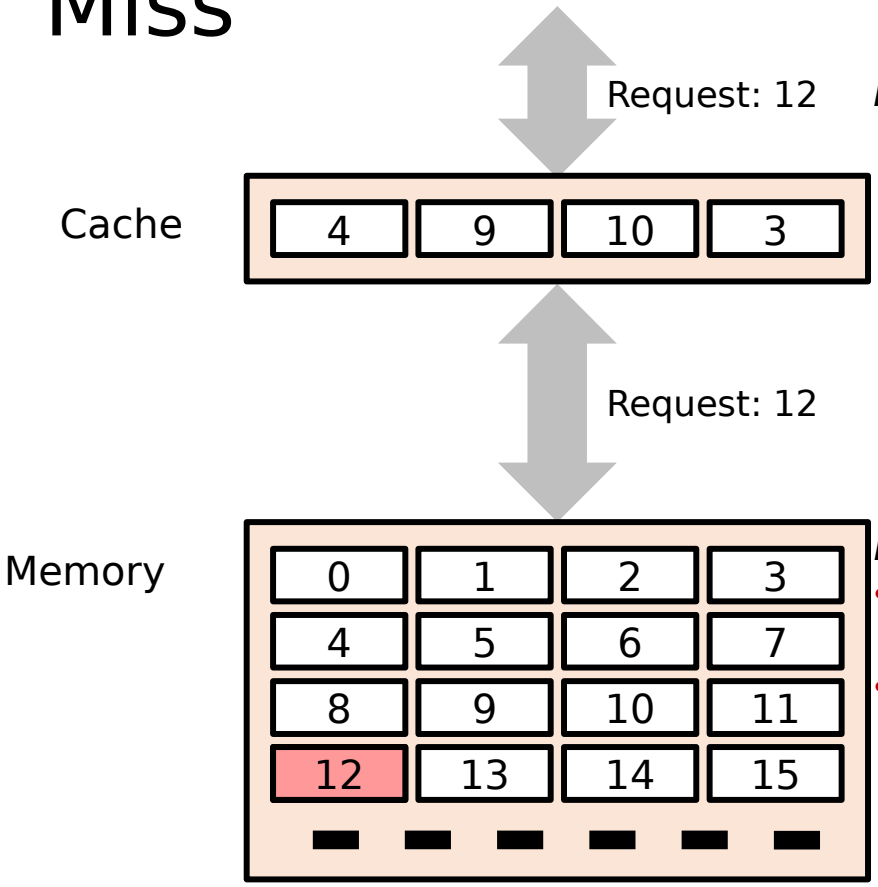
***Block b is not in cache:***
***Miss!***

***Block b is fetched from*** *memory*

***Block b is stored in cache***

- Placement policy: determines where b goes
- Replacement policy: determines which block gets evicted (victim)

# General Cache Concepts: Miss




***Data in block b is needed***

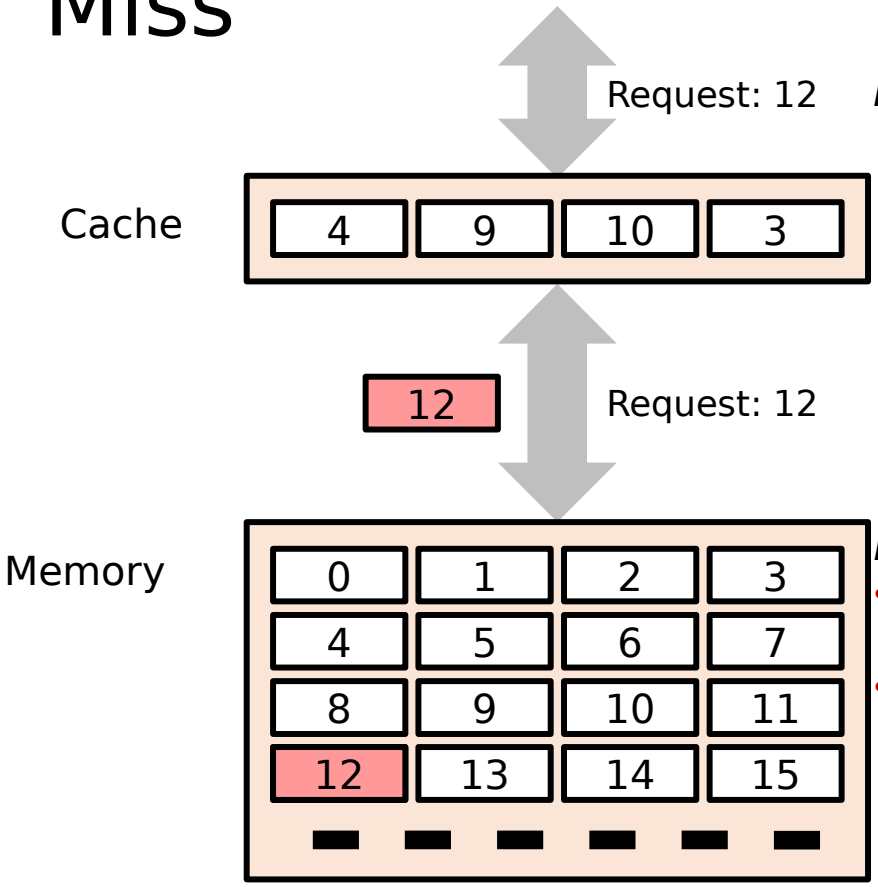
***Block b is not in cache:***
***Miss!***

***Block b is fetched from*** *memory*

***Block b is stored in cache***

- Placement policy: determines where b goes
- Replacement policy: determines which block gets evicted (victim)

# General Cache Concepts: Miss




***Data in block b is needed***

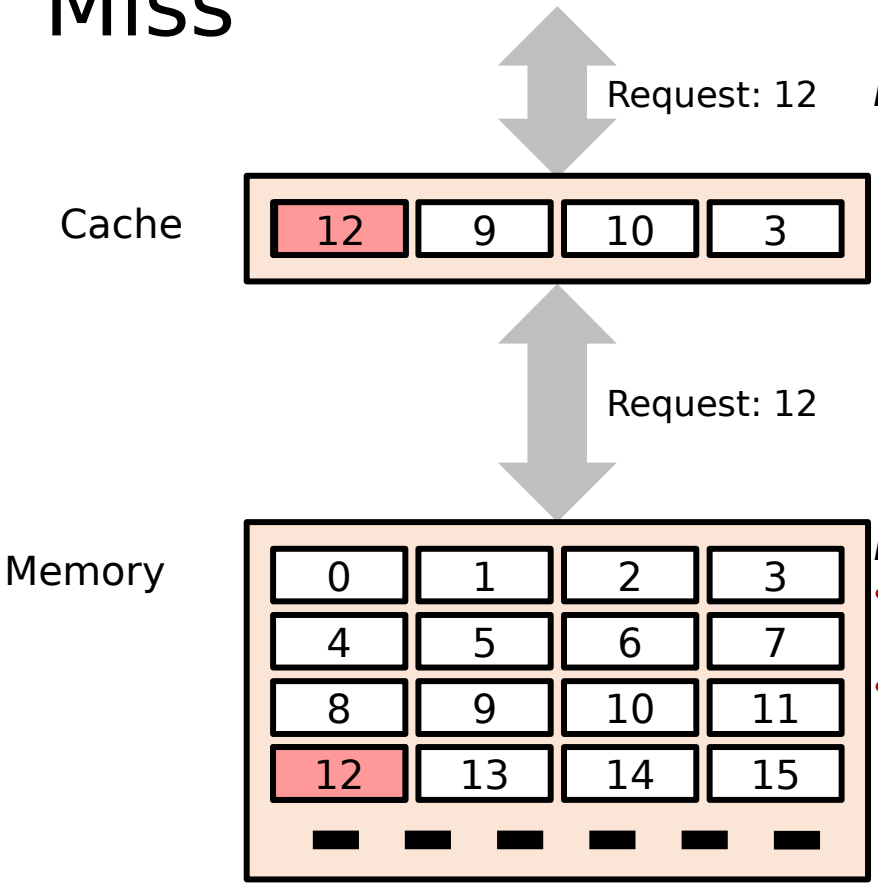
***Block b is not in cache:***
***Miss!***

***Block b is fetched from*** *memory*

***Block b is stored in cache***

- Placement policy: determines where b goes
- Replacement policy: determines which block gets evicted (victim)

# General Cache Concepts: Miss




***Data in block b is needed***

***Block b is not in cache:***
***Miss!***

***Block b is fetched from*** *memory*

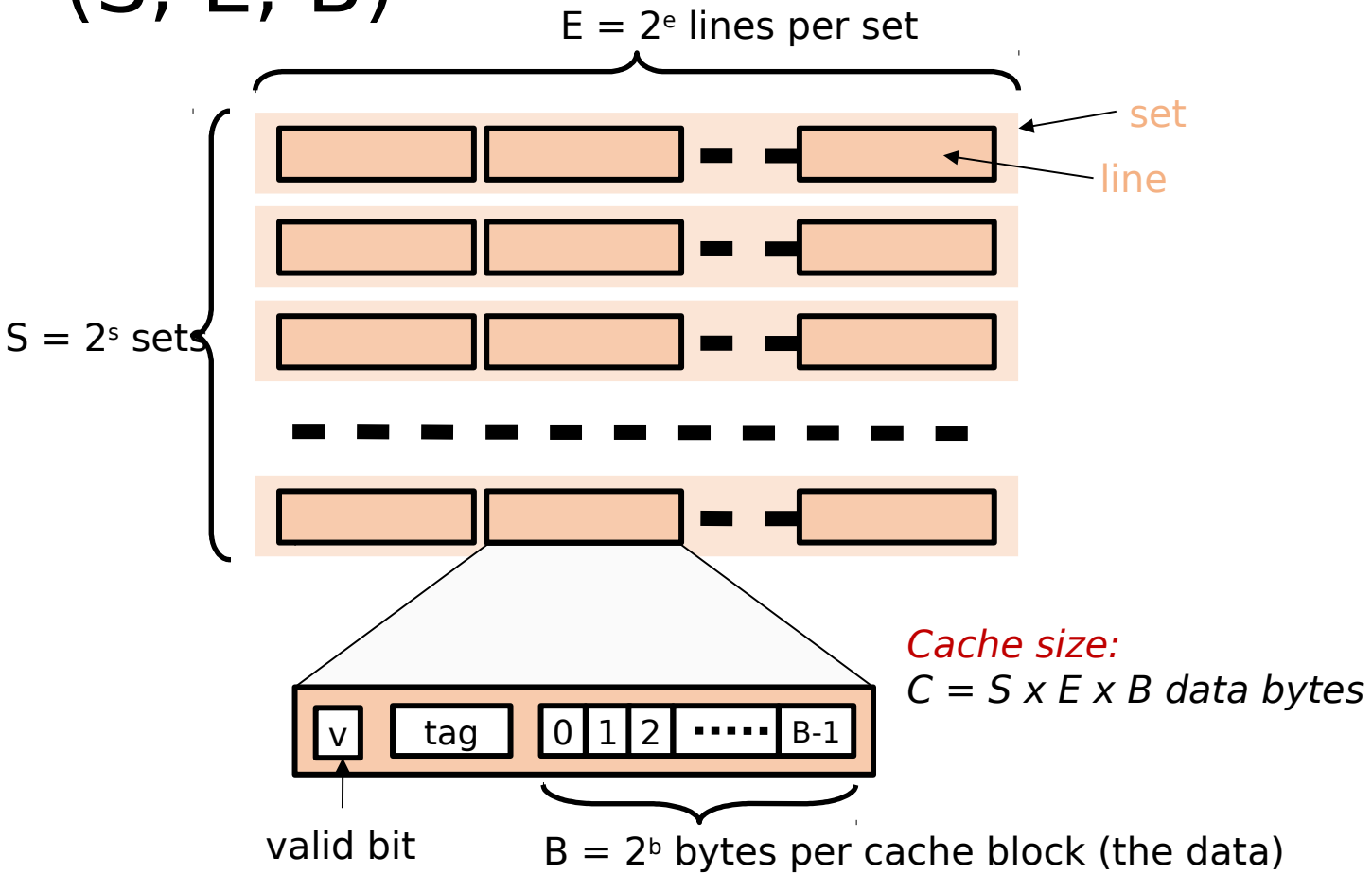
***Block b is stored in cache***

- Placement policy: determines where b goes
- Replacement policy: determines which block gets evicted (victim)

# General Caching Concepts: Types of Cache Misses

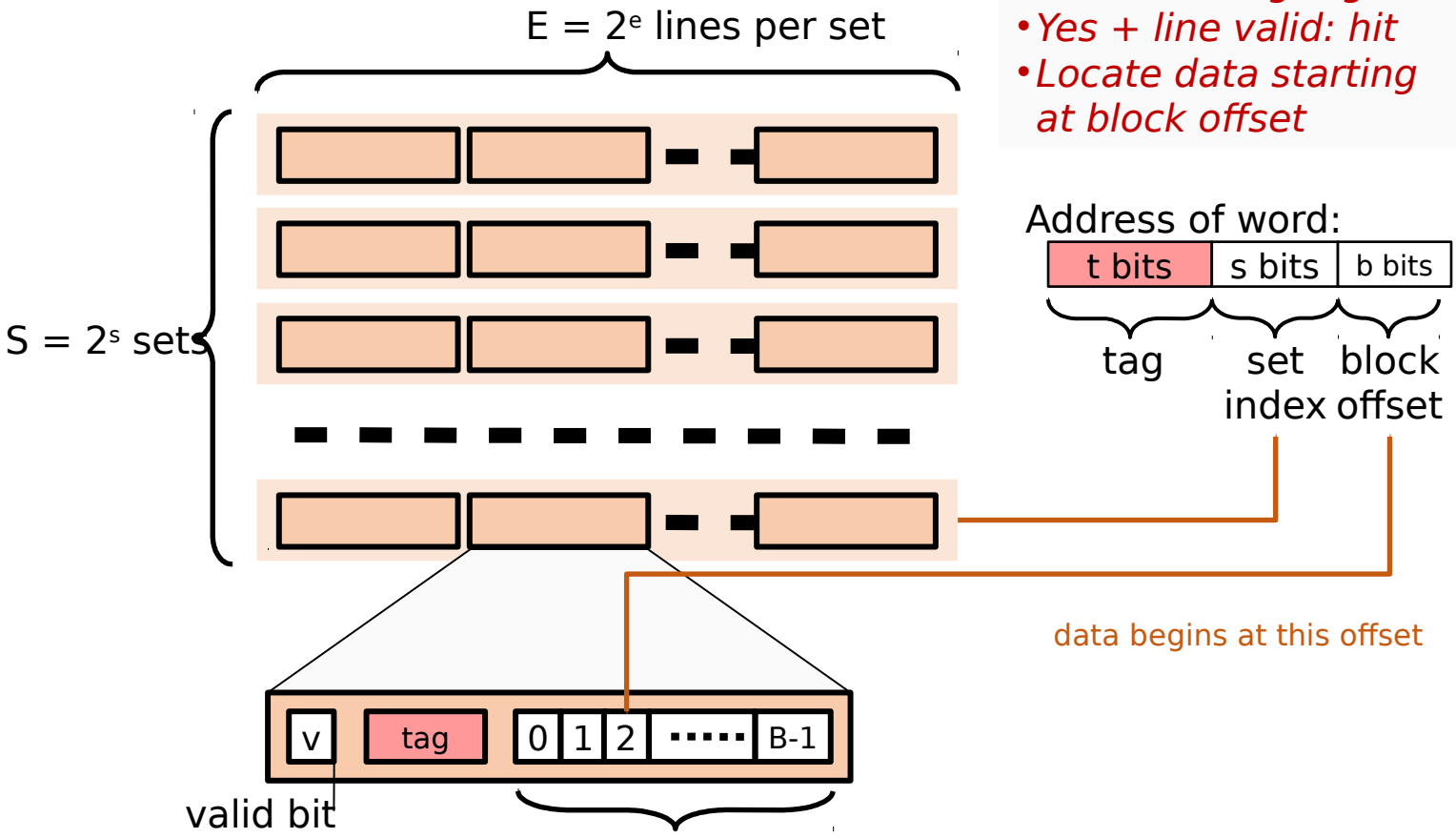
- Cold (compulsory) miss
  - Cold misses occur because the cache is empty.
- Conflict miss
  - Most caches limit blocks at level k+1 to a small subset (sometimes a singleton) of the block positions at level k.
    - E.g. Block i at level k+1 must be placed in block (i mod 4) at level k.
  - Conflict misses occur when the level k cache is large enough, but multiple data objects all map to the same level k block.
    - E.g. Referencing blocks 0, 8, 0, 8, 0, 8, ... would miss every time.
- Capacity miss
  - Occurs when the set of active cache blocks (working set) is larger than the cache.

# General Cache Organization (S, E, B)



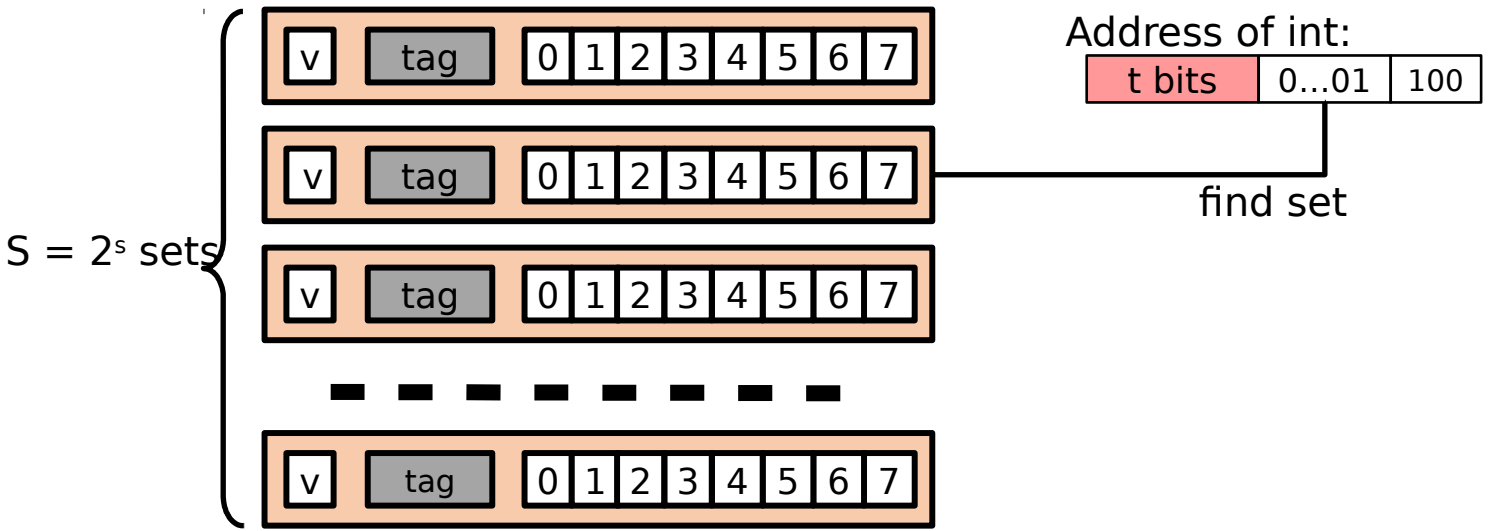
# Cache Read

- *Locate set*
- *Check if any line in set has matching tag*
- *Yes + line valid: hit*
- *Locate data starting at block offset*



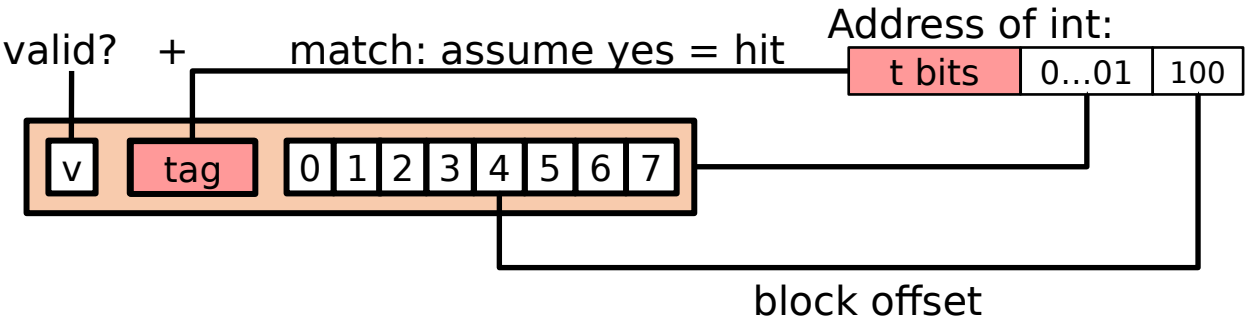
# Example: Direct Mapped Cache (E = 1)

Direct mapped: One line per set
Assume: cache block size 8 bytes



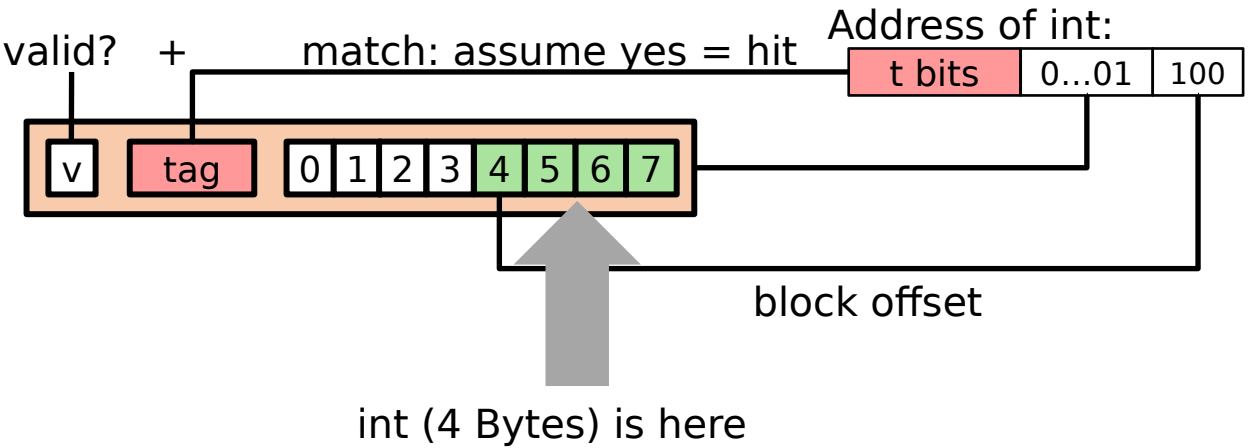
# Example: Direct Mapped Cache (E = 1)

Direct mapped: One line per set
Assume: cache block size 8 bytes



# Example: Direct Mapped Cache (E = 1)

Direct mapped: One line per set
Assume: cache block size 8 bytes




If tag doesn't match: old line is evicted and replaced

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses), B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

0 [$0000_2$],
1 [$0001_2$],
7 [$0111_2$],
8 [$1000_2$],
0 [$0000_2$]

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 0 | ? | ? |
| Set 1 | | | |
| Set 2 | | | |
| Set 3 | | | |

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses), B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$0000_2$], | miss |
| 1 | [$0001_2$], | |
| 7 | [$0111_2$], | |
| 8 | [$1000_2$], | |
| 0 | [$0000_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 0 | ? | ? |
| Set 1 | | | |
| Set 2 | | | |
| Set 3 | | | |

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses), B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$0\underline{00}0_2$], | miss |
| 1 | [$0\underline{00}1_2$], | |
| 7 | [$0\underline{11}1_2$], | |
| 8 | [$1\underline{00}0_2$], | |
| 0 | [$0\underline{00}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 0 | M[0-1] |
| Set 1 | | | |
| Set 2 | | | |
| Set 3 | | | |

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses), B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$0\underline{00}0_2$], | miss |
| 1 | [$0\underline{00}1_2$], | hit |
| 7 | [$0\underline{11}1_2$], | |
| 8 | [$1\underline{00}0_2$], | |
| 0 | [$0\underline{00}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 0 | M[0-1] |
| Set 1 | | | |
| Set 2 | | | |
| Set 3 | | | |

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses), B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$0\underline{00}0_2$], | miss |
| 1 | [$0\underline{00}1_2$], | hit |
| 7 | [$0\underline{11}1_2$], | miss |
| 8 | [$1\underline{00}0_2$], | |
| 0 | [$0\underline{00}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 0 | M[0-1] |
| Set 1 | | | |
| Set 2 | | | |
| Set 3 | | | |

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses), B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$0000_2$], | miss |
| 1 | [$0001_2$], | hit |
| 7 | [$0111_2$], | miss |
| 8 | [$1000_2$], | |
| 0 | [$0000_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 0 | M[0-1] |
| Set 1 | | | |
| Set 2 | | | |
| Set 3 | 1 | 0 | M[6-7] |

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses), B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$0\underline{00}0_2$], | miss |
| 1 | [$0\underline{00}1_2$], | hit |
| 7 | [$0\underline{11}1_2$], | miss |
| 8 | [$1\underline{00}0_2$], | miss |
| 0 | [$0\underline{00}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 0 | M[0-1] |
| Set 1 | | | |
| Set 2 | | | |
| Set 3 | 1 | 0 | M[6-7] |

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses), B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$0\underline{00}0_2$], | miss |
| 1 | [$0\underline{00}1_2$], | hit |
| 7 | [$0\underline{11}1_2$], | miss |
| 8 | [$1\underline{00}0_2$], | miss |
| 0 | [$0\underline{00}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 1 | M[8-9] |
| Set 1 | | | |
| Set 2 | | | |
| Set 3 | 1 | 0 | M[6-7] |

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses), B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$0\underline{00}0_2$], | miss |
| 1 | [$0\underline{00}1_2$], | hit |
| 7 | [$0\underline{11}1_2$], | miss |
| 8 | [$1\underline{00}0_2$], | miss |
| 0 | [$0\underline{00}0_2$] | miss |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 1 | M[8-9] |
| Set 1 | | | |
| Set 2 | | | |
| Set 3 | 1 | 0 | M[6-7] |

# Direct-Mapped Cache Simulation

| t=1 | s=2 | b=1 |
|---|---|---|
| x | xx | x |

M=16 bytes (4-bit addresses), B=2 bytes/block, S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

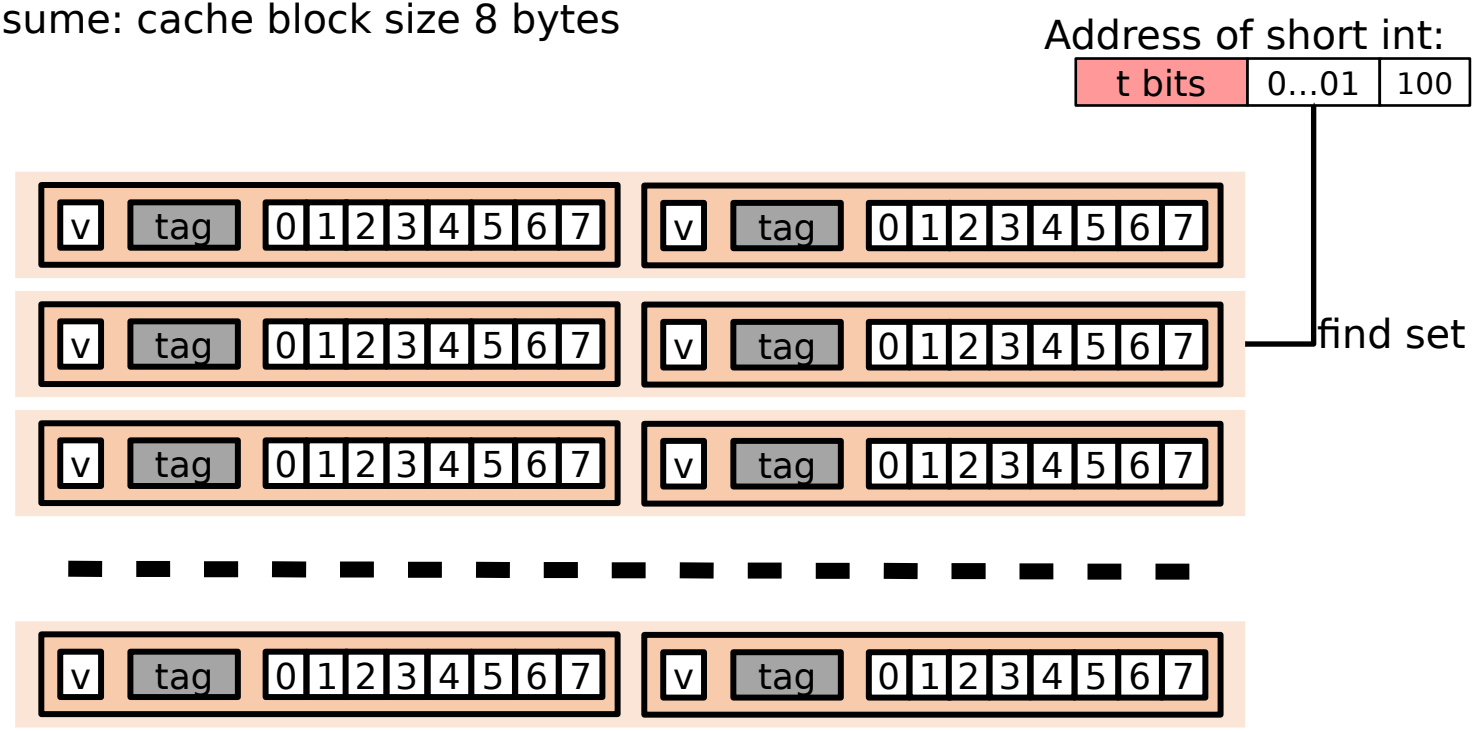
| | | |
|---|---|---|
| 0 | [$0\underline{00}0_2$], | miss |
| 1 | [$0\underline{00}1_2$], | hit |
| 7 | [$0\underline{11}1_2$], | miss |
| 8 | [$1\underline{00}0_2$], | miss |
| 0 | [$0\underline{00}0_2$] | miss |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 0 | M[0-1] |
| Set 1 | | | |
| Set 2 | | | |
| Set 3 | 1 | 0 | M[6-7] |

# E-way Set Associative Cache (Here: E = 2)

E = 2: Two lines per set

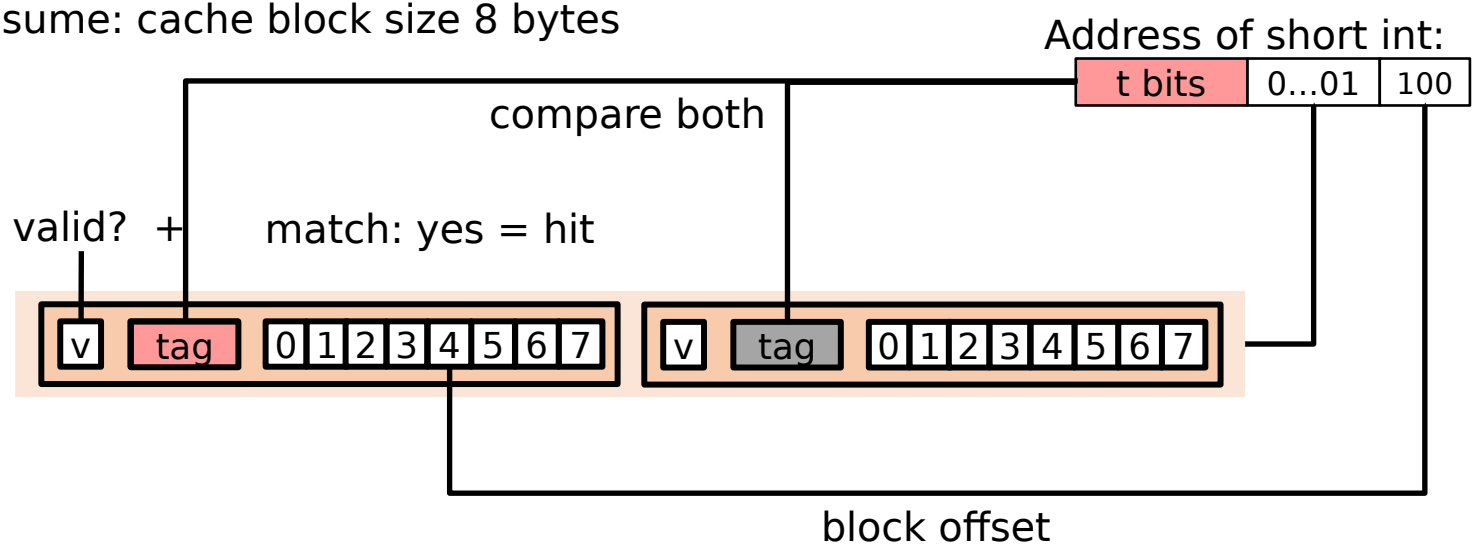
Assume: cache block size 8 bytes



# E-way Set Associative Cache (Here: E = 2)

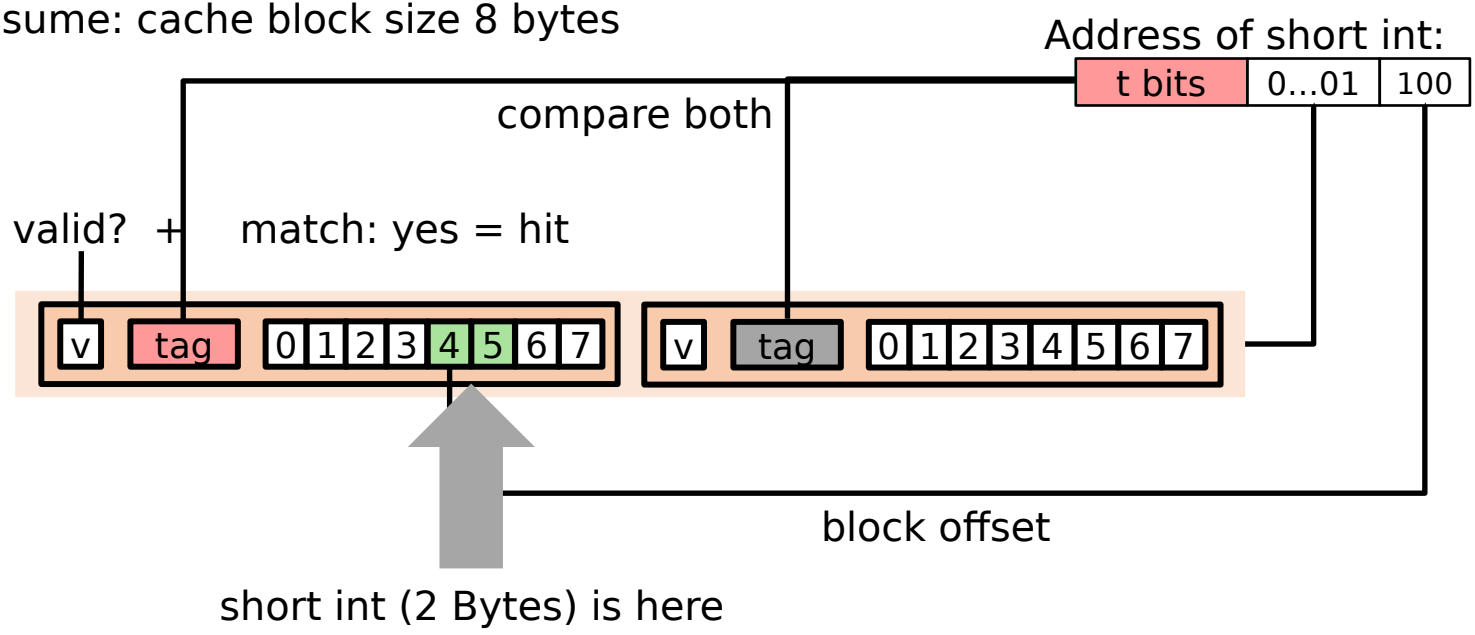
E = 2: Two lines per set

Assume: cache block size 8 bytes



# E-way Set Associative Cache (Here: E = 2)

E = 2: Two lines per set
Assume: cache block size 8 bytes




No match:

- One line in set is selected for eviction and replacement
- Replacement policies: random, least recently used (LRU), …

# 2-Way Set Associative Cache Simulation

| t=2 | s=1 | b=1 |
|---|---|---|
| xx | x | x |

M=16 byte addresses,
B=2 bytes/block, S=2 sets, E=2 blocks/set
Address trace (reads, one byte per read):

0 [$00\underline{0}0_2$],
1 [$00\underline{0}1_2$],
7 [$01\underline{1}1_2$],
8 [$10\underline{0}0_2$],
0 [$00\underline{0}0_2$]

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 0 | ? | ? |
| | 0 | | |
| Set 1 | 0 | | |
| | 0 | | |

# 2-Way Set Associative Cache Simulation

| t=2 | s=1 | b=1 |
|---|---|---|
| xx | x | x |

M=16 byte addresses,
B=2 bytes/block, S=2 sets, E=2 blocks/set
Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$00\underline{0}0_2$], | miss |
| 1 | [$00\underline{0}1_2$], | |
| 7 | [$01\underline{1}1_2$], | |
| 8 | [$10\underline{0}0_2$], | |
| 0 | [$00\underline{0}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 0 | ? | ? |
| | 0 | | |
| Set 1 | 0 | | |
| | 0 | | |

# 2-Way Set Associative Cache Simulation

| t=2 | s=1 | b=1 |
|---|---|---|
| xx | x | x |

M=16 byte addresses,
B=2 bytes/block, S=2 sets, E=2 blocks/set
Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$00\underline{0}0_2$], | miss |
| 1 | [$00\underline{0}1_2$], | |
| 7 | [$01\underline{1}1_2$], | |
| 8 | [$10\underline{0}0_2$], | |
| 0 | [$00\underline{0}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 00 | M[0-1] |
| | 0 | | |
| Set 1 | 0 | | |
| | 0 | | |

# 2-Way Set Associative Cache Simulation

| t=2 | s=1 | b=1 |
|---|---|---|
| xx | x | x |

M=16 byte addresses,
B=2 bytes/block, S=2 sets, E=2 blocks/set
Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$0000_2$], | miss |
| 1 | [$0001_2$], | hit |
| 7 | [$0111_2$], | |
| 8 | [$1000_2$], | |
| 0 | [$0000_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 00 | M[0-1] |
| | 0 | | |
| Set 1 | 0 | | |
| | 0 | | |

# 2-Way Set Associative Cache Simulation

| t=2 | s=1 | b=1 |
|---|---|---|
| xx | x | x |

M=16 byte addresses,
B=2 bytes/block, S=2 sets, E=2 blocks/set
Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$00\underline{0}0_2$], | miss |
| 1 | [$00\underline{0}1_2$], | hit |
| 7 | [$01\underline{1}1_2$], | miss |
| 8 | [$10\underline{0}0_2$], | |
| 0 | [$00\underline{0}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 00 | M[0-1] |
| | 0 | | |
| Set 1 | 0 | | |
| | 0 | | |

# 2-Way Set Associative Cache Simulation

| t=2 | s=1 | b=1 |
|---|---|---|
| xx | x | x |

M=16 byte addresses,
B=2 bytes/block, S=2 sets, E=2 blocks/set
Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$00\underline{0}0_2$], | miss |
| 1 | [$00\underline{0}1_2$], | hit |
| 7 | [$01\underline{1}1_2$], | miss |
| 8 | [$10\underline{0}0_2$], | |
| 0 | [$00\underline{0}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 00 | M[0-1] |
| | 0 | | |
| Set 1 | 1 | 01 | M[6-7] |
| | 0 | | |

# 2-Way Set Associative Cache Simulation

| t=2 | s=1 | b=1 |
|---|---|---|
| xx | x | x |

M=16 byte addresses,
B=2 bytes/block, S=2 sets, E=2 blocks/set
Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$00\underline{0}0_2$], | miss |
| 1 | [$00\underline{0}1_2$], | hit |
| 7 | [$01\underline{1}1_2$], | miss |
| 8 | [$10\underline{0}0_2$], | miss |
| 0 | [$00\underline{0}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 00 | M[0-1] |
| | 0 | | |
| Set 1 | 1 | 01 | M[6-7] |
| | 0 | | |

# 2-Way Set Associative Cache Simulation

| t=2 | s=1 | b=1 |
|---|---|---|
| xx | x | x |

M=16 byte addresses,
B=2 bytes/block, S=2 sets, E=2 blocks/set
Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$00\underline{0}0_2$], | miss |
| 1 | [$00\underline{0}1_2$], | hit |
| 7 | [$01\underline{1}1_2$], | miss |
| 8 | [$10\underline{0}0_2$], | miss |
| 0 | [$00\underline{0}0_2$] | |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 00 | M[0-1] |
| | 1 | 10 | M[8-9] |
| Set 1 | 1 | 01 | M[6-7] |
| | 0 | | |

# 2-Way Set Associative Cache Simulation

| t=2 | s=1 | b=1 |
|---|---|---|
| xx | x | x |

M=16 byte addresses,
B=2 bytes/block, S=2 sets, E=2 blocks/set
Address trace (reads, one byte per read):

| | | |
|---|---|---|
| 0 | [$00\underline{0}0_2$], | miss |
| 1 | [$00\underline{0}1_2$], | hit |
| 7 | [$01\underline{1}1_2$], | miss |
| 8 | [$10\underline{0}0_2$], | miss |
| 0 | [$00\underline{0}0_2$] | hit |

| | v | Tag | Block |
|---|---|---|---|
| Set 0 | 1 | 00 | M[0-1] |
| | 1 | 10 | M[8-9] |
| Set 1 | 1 | 01 | M[6-7] |
| | 0 | | |

# Cache Example

**Practice Problem 6.9** (solution page 699)

The following table gives the parameters for a number of different caches. For each cache, determine the number of cache sets ($S$), tag bits ($t$), set index bits ($s$), and block offset bits ($b$).

| Cache | $m$ | $C$ | $B$ | $E$ | $S$ | $t$ | $s$ | $b$ |
|---|---|---|---|---|---|---|---|---|
| 1. | 32 | 1,024 | 4 | 1 | | | | |
| 2. | 32 | 1,024 | 8 | 4 | | | | |
| 3. | 32 | 1,024 | 32 | 32 | | | | |

| Parameter | Description |
|---|---|
| Fundamental parameters | |
| $S = 2^s$ | Number of sets |
| $E$ | Number of lines per set |
| $B = 2^b$ | Block size (bytes) |
| $m = \log_2(M)$ | Number of physical (main memory) address bits |
| Derived quantities | |
| $M = 2^m$ | Maximum number of unique memory addresses |
| $s = \log_2(S)$ | Number of *set index bits* |
| $b = \log_2(B)$ | Number of *block offset bits* |
| $t = m - (s + b)$ | Number of *tag bits* |
| $C = B \times E \times S$ | Cache size (bytes), not including overhead such as the valid and tag bits |

## Practice Problem 6.9 (solution page 699)

The following table gives the parameters for a number of different caches. For each cache, determine the number of cache sets ($S$), tag bits ($t$), set index bits ($s$), and block offset bits ($b$).

| Cache | $m$ | $C$ | $B$ | $E$ | $S$ | $t$ | $s$ | $b$ |
|---|---|---|---|---|---|---|---|---|
| 1. | 32 | 1,024 | 4 | 1 | 256 | 22 | 8 | 2 |
| 2. | 32 | 1,024 | 8 | 4 | | | | |
| 3. | 32 | 1,024 | 32 | 32 | | | | |

| Parameter | Description |
|---|---|
| Fundamental parameters | |
| $S = 2^s$ | Number of sets |
| $E$ | Number of lines per set |
| $B = 2^b$ | Block size (bytes) |
| $m = \log_2(M)$ | Number of physical (main memory) address bits |
| Derived quantities | |
| $M = 2^m$ | Maximum number of unique memory addresses |
| $s = \log_2(S)$ | Number of *set index bits* |
| $b = \log_2(B)$ | Number of *block offset bits* |
| $t = m - (s + b)$ | Number of *tag bits* |
| $C = B \times E \times S$ | Cache size (bytes), not including overhead such as the valid and tag bits |

## Practice Problem 6.9 (solution page 699)

The following table gives the parameters for a number of different caches. For each cache, determine the number of cache sets ($S$), tag bits ($t$), set index bits ($s$), and block offset bits ($b$).

| Cache | $m$ | $C$ | $B$ | $E$ | $S$ | $t$ | $s$ | $b$ |
|---|---|---|---|---|---|---|---|---|
| 1. | 32 | 1,024 | 4 | 1 | 256 | 22 | 8 | 2 |
| 2. | 32 | 1,024 | 8 | 4 | 32 | 24 | 5 | 3 |
| 3. | 32 | 1,024 | 32 | 32 | | | | |

| Parameter | Description |
|---|---|
| Fundamental parameters | |
| $S = 2^s$ | Number of sets |
| $E$ | Number of lines per set |
| $B = 2^b$ | Block size (bytes) |
| $m = \log_2(M)$ | Number of physical (main memory) address bits |
| Derived quantities | |
| $M = 2^m$ | Maximum number of unique memory addresses |
| $s = \log_2(S)$ | Number of *set index bits* |
| $b = \log_2(B)$ | Number of *block offset bits* |
| $t = m - (s + b)$ | Number of *tag bits* |
| $C = B \times E \times S$ | Cache size (bytes), not including overhead such as the valid and tag bits |

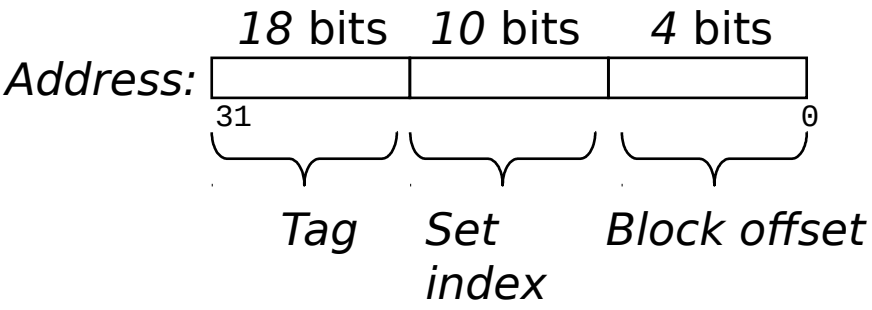
## Practice Problem 6.9 (solution page 699)

The following table gives the parameters for a number of different caches. For each cache, determine the number of cache sets ($S$), tag bits ($t$), set index bits ($s$), and block offset bits ($b$).

| Cache | $m$ | $C$ | $B$ | $E$ | $S$ | $t$ | $s$ | $b$ |
|---|---|---|---|---|---|---|---|---|
| 1. | 32 | 1,024 | 4 | 1 | 256 | 22 | 8 | 2 |
| 2. | 32 | 1,024 | 8 | 4 | 32 | 24 | 5 | 3 |
| 3. | 32 | 1,024 | 32 | 32 | 1 | 27 | 0 | 5 |

# Calculating Cache Parameters

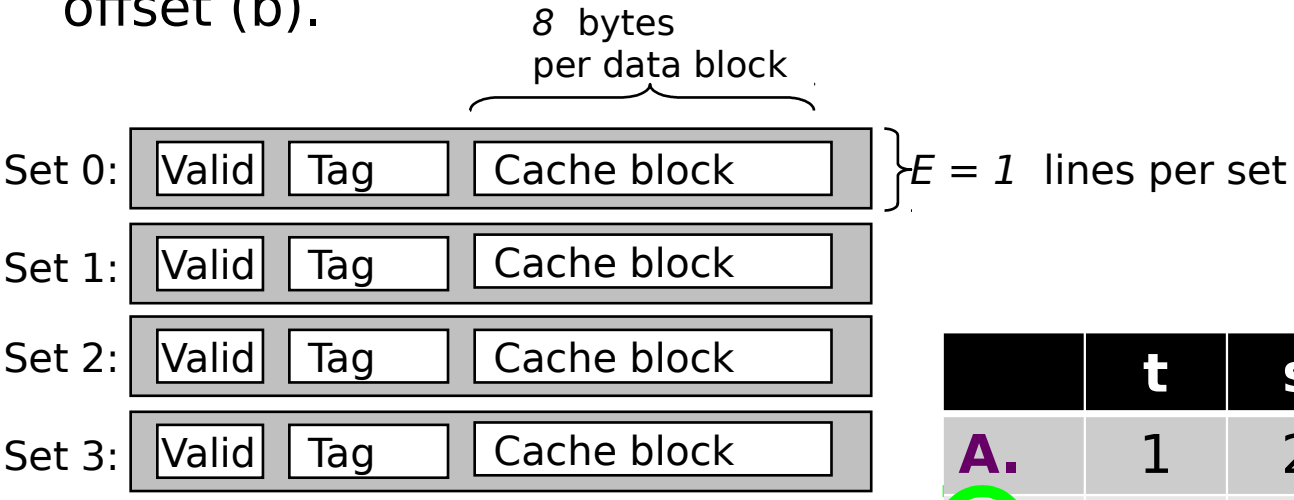
Given the following address partition, how many `int` values will fit in a single data block?

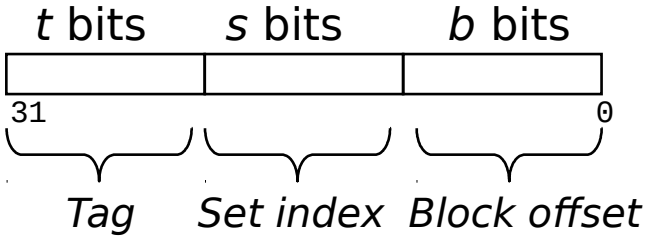



| | # of int in block |
|---|---|
| **A.** | 0 |
| **B.** | 1 |
| **C.** | 2 |
| **D.** | 4 |
| **E.** | Unknown: We need more info |

# Direct-Mapped Cache Example

Assuming a 32-bit address (i.e. m=32), how many bits are used for tag (t), set index (s), and block offset (b).

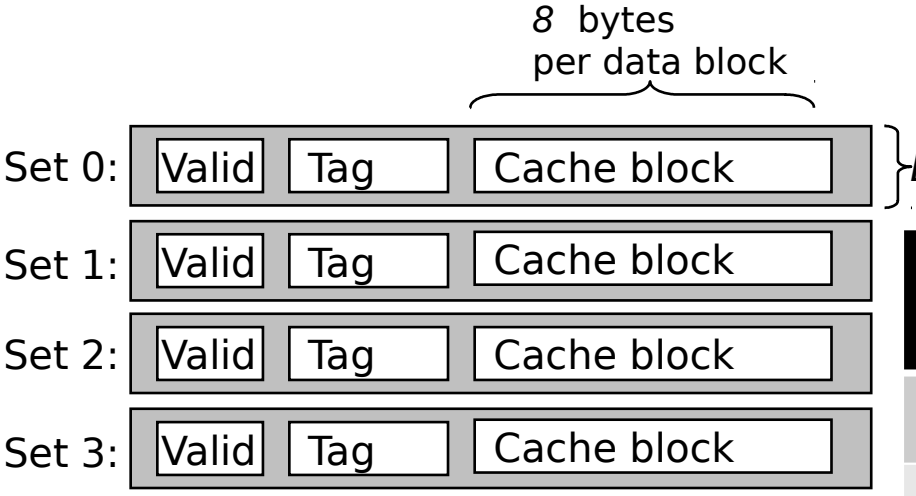


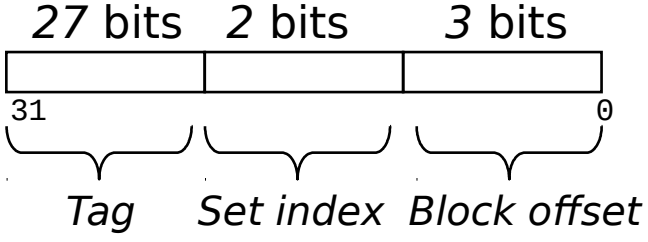




| | t | s | b |
|---|---|---|---|
| A. | 1 | 2 | 3 |
| B. | 27 | 2 | 3 |
| C. | 25 | 4 | 3 |
| D. | 1 | 4 | 8 |
| E. | 20 | 4 | 8 |

# Which Set Is it?

Which set may the address **0xFA1C** be located in?

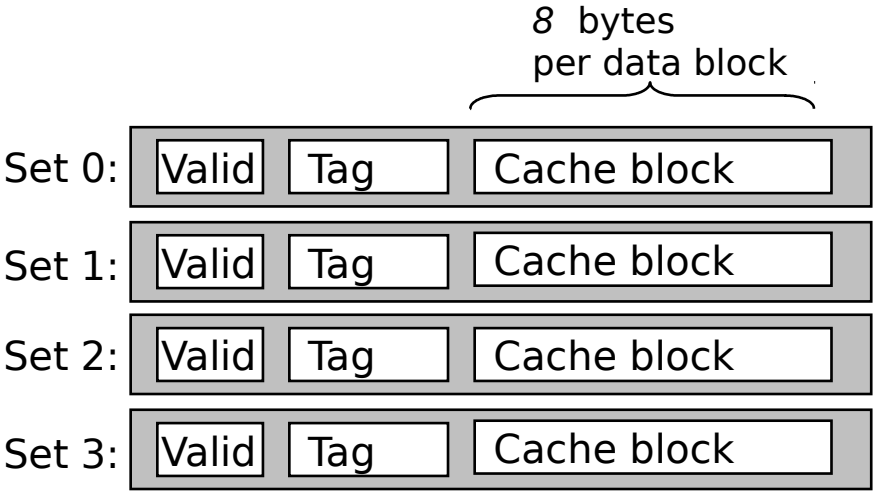


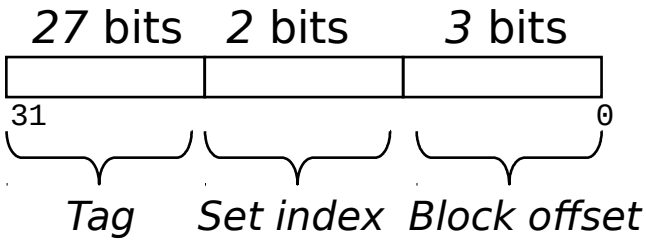

| | Set # for 0xFA1C |
|---|---|
| **A.** | 0 |
| **B.** | 1 |
| **C.** | 2 |
| **D.** | 3 |
| **E.** | More than one of the above |

# Cache Block Range

What range of addresses will be in the same block as address **0xFA1C**?




| | Addr. Range |
|---|---|
| **A.** | 0xFA1C |
| **B.** | 0xFA1C – 0xFA23 |
| **C.** | 0xFA1C – 0xFA1F |
| **D.** | 0xFA18 – 0xFA1F |
| **E.** | It depends on the access size (byte, word, etc) |



# Cache Misses

If N = 16, how many bytes does the loop access of a?

```
int foo(int* a, int N)
{
    int i, sum = 0;
    for(i = 0; i < N; i++)
        sum += a[i];
    return sum;
}
```

| | Accessed Bytes |
|---|---|
| **A** | 4 |
| **B** | 16 |
| **C** | 64 |
| **D** | 256 |

# Cache Misses

If there is a 64B cache with 8 bytes per block and two blocks per set, how many misses if foo is called twice?
N still equals 16

```
int foo(int* a, int N)
{
    int i, sum = 0;
    for(i = 0; i < N; i++)
        sum += a[i];
    return sum;
}
```

| | Misses |
|---|---|
| **A** | 0 |
| **B** | 8 |
| **C** | 12 |
| **D** | 14 |
| **E** | 16 |